Condition Variables - Thread Synchronization and Event Notification

Theory, Edge Cases & Code Examples

- ✓ Condition Variable Overview
- ✓ The Wait-Notify Mechanism
- ✓ Why std::unique_lock is Required
- ✓ Notification Strategies
- ✓ Producer-Consumer Pattern

→ Swipe to learn →

Condition Variables - Thread Synchronization and Event Notification

Theory

📌 1. Condition Variable Overview

Definition:

- Synchronization primitive enabling threads to wait for specific conditions without busy-waiting
- Works with a mutex to coordinate thread communication
- Defined in `<condition_variable>` header (C++11)

Core Mechanism:

Component	Role
Mutex	Protects the shared state (the condition itself)
Condition Variable	Coordinates waiting and waking of threads
Predicate	Boolean condition that determines when thread should wake

Basic Operations:

```
std::mutex mtx;
std::condition_variable cv;
bool ready = false;

// Waiter thread:
std::unique_lock<std::mutex> lock(mtx);
cv.wait(lock, []{ return ready; });  // Wait until ready == true

// Notifier thread:
{
    std::lock_guard<std::mutex> lock(mtx);
    ready = true;
}
cv.notify_one();  // Wake one waiting thread
```

Condition Variables - Thread Synchronization and Event Notification

Theory

📌 2. The Wait-Notify Mechanism (1/2)

wait() Operation Steps:

Step	Action	Details
1	Check predicate	If already true, return immediately (no wait)
2	Unlock mutex	Atomically release lock
3	Block thread	OS removes thread from scheduler (sleep)
4	Wake on notification	Triggered by notify_one/notify_all or spuriously
5	Relock mutex	Atomically reacquire lock before returning
6	Recheck predicate	Loop back if still false (spurious wakeup)
7	Return	Predicate guaranteed true

Predicate Form:

```
// ✅ Best practice - handles spurious wakeups automatically
cv.wait(lock, []{ return ready; });

// Equivalent manual loop:
while (!ready) {
    cv.wait(lock);  // Without predicate
}
```

Condition Variables - Thread Synchronization and Event Notification

Theory

📌 2. The Wait-Notify Mechanism (2/2)

Spurious Wakeups:

Concept	Explanation
What	Thread wakes from wait() without being notified
Why	Implementation optimization, signal handling, OS scheduling
Protection	Always use predicate form of wait()
Consequence without predicate	Thread might proceed when condition still false → incorrect behavior

Condition Variables - Thread Synchronization and Event Notification

Theory

📌 3. Why std::unique_lock is Required

Lock Type Comparison:

Feature	std::lock_guard	std::unique_lock	Required for CV?
Manual unlock	❌ No	✅ Yes	✅ Essential
Manual relock	❌ No	✅ Yes	✅ Essential
Overhead	Zero	Minimal	Worth it for flexibility
Use with wait()	❌ Compilation error	✅ Works	-

Why Flexibility Matters:

```
void consumer() {
    std::unique_lock<std::mutex> lock(mtx);

    // wait() needs to:
    // 1. Unlock mtx (to let producer modify condition)
    // 2. Sleep thread
    // 3. Relock mtx (when woken)
    // → unique_lock provides unlock()/lock() methods

    cv.wait(lock, []{ return !queue.empty(); });
    process(queue.front());
}
```

Condition Variables - Thread Synchronization and Event Notification

Theory

📌 4. Notification Strategies

notify_one() vs notify_all():

Operation	Wakes	Use When	Example
notify_one()	One thread	Only one thread can proceed	Single item added to queue
notify_all()	All threads	Multiple threads can proceed or broadcast event	Shutdown signal, configuration change

Notification Timing:

```
// Option 1: Notify inside lock
{
    std::lock_guard<std::mutex> lock(mtx);
    ready = true;
    cv.notify_one();  // Inside lock
}

// Option 2: Notify outside lock (often better performance)
{
    std::lock_guard<std::mutex> lock(mtx);
    ready = true;
}  // Lock released here
cv.notify_one();  // ✅ Outside lock - woken thread can acquire immediately
```

Performance Trade-off:

Approach	Pros	Cons
Notify inside lock	Simple, all state changes atomic	Woken thread blocks again waiting for lock release
Notify outside lock	Woken thread acquires lock faster	Slightly more complex (separate scope)

Condition Variables - Thread Synchronization and Event Notification

Theory

📌 5. Producer-Consumer Pattern

Classic Coordination:

```
std::mutex mtx;
std::condition_variable cv_not_empty;
std::queue<Item> queue;

// Producer:
{
    std::lock_guard<std::mutex> lock(mtx);
    queue.push(item);
}
cv_not_empty.notify_one();  // Wake consumer

// Consumer:
std::unique_lock<std::mutex> lock(mtx);
cv_not_empty.wait(lock, []{ return !queue.empty(); });
Item item = queue.front();
queue.pop();
```

Bounded Buffer (Two Condition Variables):

Condition Variable	Waits When	Woken When
cv_not_full	Buffer full (producer waits)	Consumer removes item
cv_not_empty	Buffer empty (consumer waits)	Producer adds item

Benefits vs Busy-Waiting:

Approach	CPU Usage When Waiting	Latency	Power Consumption
Busy-waiting	100% (spinning in loop)	Low (immediate detection)	High
Condition Variable	0% (thread sleeping)	Low (instant wake on notify)	Minimal

---

📌 6. Real-World: Autonomous Driving Systems (1/3)

Perception Pipeline Example:

Component	Thread Role	CV Usage
Camera Capture (30Hz)	Producer	Pushes frames to queue, notifies processing thread
Perception Processing	Consumer	Waits for frames, wakes instantly on arrival
Planning (10Hz)	Consumer	Waits for updated world model from sensor fusion
Control (100Hz)	Consumer	Waits for new path commands from planner

Condition Variables - Thread Synchronization and Event Notification

Theory

📌 6. Real-World: Autonomous Driving Systems (2/3)

Critical Requirements:

```
// Camera thread:
void camera_capture_thread() {
    while (!shutdown) {
        Frame frame = capture();  // 30 FPS

        {
            std::lock_guard<std::mutex> lock(mtx);
            frame_queue.push(frame);
        }
        cv_frame_ready.notify_one();  // Instant wake
    }
}

// Perception thread:
void perception_thread() {
    while (true) {
        Frame frame;

        {
            std::unique_lock<std::mutex> lock(mtx);
            cv_frame_ready.wait(lock, []{ return shutdown || !frame_queue.empty(); });

            if (shutdown && frame_queue.empty()) break;

            frame = frame_queue.front();
            frame_queue.pop();
        }

        process_frame(frame);  // Object detection, 33ms deadline
    }
}
```

📌 6. Real-World: Autonomous Driving Systems (3/3)

Why This Matters:

Metric	Requirement	CV Benefit
Latency	< 5μs wake-up time	notify() wakes thread in 1-5μs
CPU Efficiency	Processing thread should sleep when idle	0% CPU when waiting vs 100% busy-wait
Real-time Deadlines	Frame processing must start immediately	Instant wake on frame arrival
Power	Automotive power budget constraints	Sleeping threads consume minimal power

Condition Variables - Thread Synchronization and Event Notification

Theory

📌 7. Common Patterns Summary

Pattern Table:

Pattern	Use Case	Key Components
Producer-Consumer	Work queue coordination	1 CV (not_empty), shared queue
Bounded Buffer	Limited capacity queue	2 CVs (not_full, not_empty)
Barrier Sync	Wait for N threads to reach point	Counter + CV + generation counter
Event Broadcast	Signal all threads (shutdown)	Flag + notify_all()
Thread Pool	Task distribution	Queue + workers waiting on CV

Best Practices:

- ✅ Always use predicate form: `cv.wait(lock, predicate)`
- ✅ Hold mutex when modifying condition
- ✅ Same mutex for condition modification and wait()
- ✅ Use `notify_one()` when only one thread can proceed
- ✅ Use `notify_all()` for broadcast events (shutdown)
- ✅ Include shutdown flag in predicates for graceful termination
- ❌ Never use `lock_guard` with wait() (compilation error)
- ❌ Never modify condition without holding mutex (data race)

Condition Variables - Thread Synchronization and Event Notification

Edge Cases

📌 1. Spurious Wakeups Without Predicate Check

```
std::mutex mtx;
std::condition_variable cv;
bool ready = false;

void consumer_unsafe() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock);  // ❌ No predicate - vulnerable to spurious wakeups

    // Might execute even if ready is still false!
    process_data();
}

void consumer_safe() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, []{ return ready; });  // ✅ Predicate protects against spurious wakeups

    // Guaranteed ready == true here
    process_data();
}
```

Spurious wakeups are allowed by the standard—implementation may wake threads for optimization or signal handling. Always use the predicate form or manual loop.

Why spurious wakeups exist: On some systems, broadcast signals can wake all threads waiting on condition variables. Rather than requiring implementations to filter these perfectly, the standard allows spurious wakeups, pushing the responsibility to user code.

Condition Variables - Thread Synchronization and Event Notification

Edge Cases

📌 2. Notify Before Wait - Lost Wakeup

```
std::mutex mtx;
std::condition_variable cv;
bool ready = false;

void producer() {
    {
        std::lock_guard<std::mutex> lock(mtx);
        ready = true;
    }
    cv.notify_one();  // ✅ Notification sent
}

void consumer() {
    std::this_thread::sleep_for(std::chrono::milliseconds(100));  // Delay

    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, []{ return ready; });  // ❌ Might have missed notification!
    // Actually, this is OK because predicate is checked first
}
```

Important: With predicates, this pattern is safe! The predicate is checked *before* waiting. If `ready == true`, `wait()` returns immediately without blocking. Without predicates (`cv.wait(lock)` only), notifications before waiting are lost, causing indefinite blocking.

Condition Variables - Thread Synchronization and Event Notification

Edge Cases

📌 3. Forgetting to Hold Lock When Modifying Condition

```
std::mutex mtx;
std::condition_variable cv;
bool ready = false;

void producer_race() {
    ready = true;  // ❌ Modified without holding lock - race condition!
    cv.notify_one();
}

void consumer() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, []{ return ready; });  // Might see inconsistent state
}

void producer_correct() {
    {
        std::lock_guard<std::mutex> lock(mtx);  // ✅ Lock protects condition
        ready = true;
    }
    cv.notify_one();  // Can notify outside lock (optional optimization)
}
```

The condition variable doesn't protect the shared state—the mutex does. Always hold the lock when modifying the condition.

Condition Variables - Thread Synchronization and Event Notification

Edge Cases

📌 4. Notify Inside vs Outside Lock

```
std::mutex mtx;
std::condition_variable cv;
bool ready = false;

void notify_inside_lock() {
    std::lock_guard<std::mutex> lock(mtx);
    ready = true;
    cv.notify_one();  // Notification while holding lock
}

void notify_outside_lock() {
    {
        std::lock_guard<std::mutex> lock(mtx);
        ready = true;
    }  // Lock released here
    cv.notify_one();  // ✅ Notification after releasing lock (often better)
}
```

Performance consideration: Notifying inside the lock means the woken thread immediately blocks again waiting for the lock to be released. Notifying outside allows the woken thread to potentially acquire the lock faster. However, both are correct—notifying outside is a performance optimization.

Condition Variables - Thread Synchronization and Event Notification

Edge Cases

📌 5. notify_one vs notify_all with Multiple Waiters

```
std::mutex mtx;
std::condition_variable cv;
std::queue<int> queue;

// Multiple consumers waiting
void consumer() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, []{ return !queue.empty(); });

    int item = queue.front();
    queue.pop();
    process(item);
}

void producer_wrong() {
    {
        std::lock_guard<std::mutex> lock(mtx);
        queue.push(42);
    }
    cv.notify_all();  // ❌ Wakes ALL consumers, but only one can pop
    // Other consumers re-block immediately (thundering herd)
}

void producer_correct() {
    {
        std::lock_guard<std::mutex> lock(mtx);
        queue.push(42);
    }
    cv.notify_one();  // ✅ Wakes one consumer (the one that can actually proceed)
}
```

Use `notify_one()` when only one thread can proceed. Use `notify_all()` when the condition might allow multiple threads to proceed, or when you can't predict which thread should wake.

Condition Variables - Thread Synchronization and Event Notification

Code Examples

📌 1. Basic Wait and Notify

This demonstrates the fundamental wait-notify pattern: worker waits for coordinator to set the ready flag.

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>

std::mutex mtx;
std::condition_variable cv;
bool ready = false;

void worker() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, []{ return ready; });  // Wait until ready == true

    std::cout << "Worker proceeding\n";
}

void coordinator() {
    std::this_thread::sleep_for(std::chrono::seconds(1));

    {
        std::lock_guard<std::mutex> lock(mtx);
        ready = true;
    }

    cv.notify_one();
    std::cout << "Coordinator notified\n";
}

int main() {
    std::thread t1(worker);
    std::thread t2(coordinator);

    t1.join();
    t2.join();
}
```

Condition Variables - Thread Synchronization and Event Notification

Code Examples

📌 2. Producer-Consumer with Single Item

Consumer waits for producer to populate data, avoiding busy-waiting.

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>

std::mutex mtx;
std::condition_variable cv;
int data = 0;
bool data_ready = false;

void producer() {
    std::this_thread::sleep_for(std::chrono::seconds(1));

    {
        std::lock_guard<std::mutex> lock(mtx);
        data = 42;
        data_ready = true;
    }

    cv.notify_one();
    std::cout << "Producer: Data produced\n";
}

void consumer() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, []{ return data_ready; });

    std::cout << "Consumer: Received data = " << data << "\n";
}

int main() {
    std::thread t_consumer(consumer);
    std::thread t_producer(producer);

    t_consumer.join();
    t_producer.join();
}
```

Condition Variables - Thread Synchronization and Event Notification

Code Examples

📌 3. Bounded Buffer Producer-Consumer (1/2)

Two condition variables coordinate bounded buffer: producers wait when full, consumers wait when empty.

```
#include <iostream>
#include <thread>
#include <queue>
#include <mutex>
#include <condition_variable>

const size_t MAX_BUFFER_SIZE = 5;

std::mutex mtx;
std::condition_variable cv_not_full;
std::condition_variable cv_not_empty;
std::queue<int> buffer;

void producer(int id, int items) {
    for (int i = 0; i < items; ++i) {
        std::unique_lock<std::mutex> lock(mtx);

        // Wait if buffer is full
        cv_not_full.wait(lock, []{ return buffer.size() < MAX_BUFFER_SIZE; });

        buffer.push(i);
        std::cout << "Producer " << id << " produced " << i
                  << " (buffer size: " << buffer.size() << ")\n";

        cv_not_empty.notify_one();  // Wake consumer
    }
}

void consumer(int id, int items) {
    for (int i = 0; i < items; ++i) {
        std::unique_lock<std::mutex> lock(mtx);

        // Wait if buffer is empty
        cv_not_empty.wait(lock, []{ return !buffer.empty(); });

        int value = buffer.front();
        buffer.pop();
        std::cout << "Consumer " << id << " consumed " << value
// ...continues on next slide
```

Condition Variables - Thread Synchronization and Event Notification

Code Examples

📌 3. Bounded Buffer Producer-Consumer (2/2)

[Code continues across slides]

```
// ...continued from previous slide
                  << " (buffer size: " << buffer.size() << ")\n";

        cv_not_full.notify_one();  // Wake producer
    }
}

int main() {
    std::thread p1(producer, 1, 10);
    std::thread c1(consumer, 1, 10);

    p1.join();
    c1.join();
}
```

Condition Variables - Thread Synchronization and Event Notification

Code Examples

📌 4. Thread-Safe Queue Class (1/2)

Reusable thread-safe queue with blocking `pop()` and timeout-based `try_pop()`.

```
#include <queue>
#include <mutex>
#include <condition_variable>
#include <optional>

template<typename T>
class ThreadSafeQueue {
private:
    mutable std::mutex mtx;
    std::condition_variable cv;
    std::queue<T> queue;

public:
    void push(T value) {
        {
            std::lock_guard<std::mutex> lock(mtx);
            queue.push(std::move(value));
        }
        cv.notify_one();
    }

    T pop() {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, [this]{ return !queue.empty(); });

        T value = std::move(queue.front());
        queue.pop();
        return value;
    }

    std::optional<T> try_pop(std::chrono::milliseconds timeout) {
        std::unique_lock<std::mutex> lock(mtx);

        if (!cv.wait_for(lock, timeout, [this]{ return !queue.empty(); })) {
            return std::nullopt;  // Timeout
        }

        T value = std::move(queue.front());
// ...continues on next slide
```

Condition Variables - Thread Synchronization and Event Notification

Code Examples

📌 4. Thread-Safe Queue Class (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        queue.pop();
        return value;
    }

    bool empty() const {
        std::lock_guard<std::mutex> lock(mtx);
        return queue.empty();
    }
};

// Usage
int main() {
    ThreadSafeQueue<int> queue;

    std::thread producer([&]{
        for (int i = 0; i < 5; ++i) {
            queue.push(i);
            std::cout << "Pushed " << i << "\n";
        }
    });

    std::thread consumer([&]{
        for (int i = 0; i < 5; ++i) {
            int value = queue.pop();
            std::cout << "Popped " << value << "\n";
        }
    });

    producer.join();
    consumer.join();
}
```

Condition Variables - Thread Synchronization and Event Notification

Code Examples

📌 5. Wait with Timeout

`wait_for()` returns false if timeout expires before condition is met.

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <chrono>

std::mutex mtx;
std::condition_variable cv;
bool ready = false;

void worker() {
    std::unique_lock<std::mutex> lock(mtx);

    // Wait for 2 seconds
    if (cv.wait_for(lock, std::chrono::seconds(2), []{ return ready; })) {
        std::cout << "Worker: Condition met\n";
    } else {
        std::cout << "Worker: Timeout\n";
    }
}

void coordinator_slow() {
    std::this_thread::sleep_for(std::chrono::seconds(3));  // Exceeds timeout

    {
        std::lock_guard<std::mutex> lock(mtx);
        ready = true;
    }
    cv.notify_one();
}

int main() {
    std::thread t1(worker);
    std::thread t2(coordinator_slow);

    t1.join();
    t2.join();
}
```

Condition Variables - Thread Synchronization and Event Notification

Code Examples

📌 6. Multiple Consumers with notify_all (1/2)

`notify_all()` wakes all waiting threads when condition allows multiple threads to proceed.

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <vector>

std::mutex mtx;
std::condition_variable cv;
bool start_processing = false;

void worker(int id) {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, []{ return start_processing; });

    std::cout << "Worker " << id << " started processing\n";
}

void coordinator() {
    std::this_thread::sleep_for(std::chrono::seconds(1));

    {
        std::lock_guard<std::mutex> lock(mtx);
        start_processing = true;
    }

    cv.notify_all();  // Wake ALL workers simultaneously
    std::cout << "Coordinator: All workers notified\n";
}

int main() {
    std::vector<std::thread> workers;
    for (int i = 0; i < 5; ++i) {
        workers.emplace_back(worker, i);
    }

    std::thread coord(coordinator);

    for (auto& t : workers) {
// ...continues on next slide
```

Condition Variables - Thread Synchronization and Event Notification

Code Examples

📌 6. Multiple Consumers with notify_all (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        t.join();
    }
    coord.join();
}
```

Condition Variables - Thread Synchronization and Event Notification

Code Examples

📌 7. Avoiding Lost Wakeups

Predicates prevent lost wakeups—condition is checked before waiting, so early notifications don't cause issues.

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>

std::mutex mtx;
std::condition_variable cv;
int counter = 0;

void early_notifier() {
    {
        std::lock_guard<std::mutex> lock(mtx);
        counter = 10;
    }
    cv.notify_one();  // Notification happens before waiter starts
    std::cout << "Notified early\n";
}

void late_waiter() {
    std::this_thread::sleep_for(std::chrono::milliseconds(100));  // Delay

    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, []{ return counter > 0; });  // ✅ Predicate checked first

    std::cout << "Waiter: counter = " << counter << "\n";  // Works despite early notification
}

int main() {
    std::thread t1(early_notifier);
    std::thread t2(late_waiter);

    t1.join();
    t2.join();
}
```

Condition Variables - Thread Synchronization and Event Notification

Code Examples

📌 8. Termination Signal for Thread Pool (1/2)

Condition variables enable graceful thread pool shutdown: workers wait for tasks or stop signal.

```
#include <iostream>
#include <thread>
#include <vector>
#include <queue>
#include <mutex>
#include <condition_variable>
#include <functional>

class SimpleThreadPool {
private:
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex mtx;
    std::condition_variable cv;
    bool stop = false;

public:
    SimpleThreadPool(size_t num_threads) {
        for (size_t i = 0; i < num_threads; ++i) {
            workers.emplace_back([this] {
                while (true) {
                    std::function<void()> task;

                    {
                        std::unique_lock<std::mutex> lock(mtx);
                        cv.wait(lock, [this]{ return stop || !tasks.empty(); });

                        if (stop && tasks.empty()) {
                            return;  // Exit thread
                        }

                        task = std::move(tasks.front());
                        tasks.pop();
                    }

                    task();  // Execute outside lock
                }
// ...continues on next slide
```

Condition Variables - Thread Synchronization and Event Notification

Code Examples

📌 8. Termination Signal for Thread Pool (2/2)

[Code continues across slides]

```
// ...continued from previous slide
            });
        }
    }

    void enqueue(std::function<void()> task) {
        {
            std::lock_guard<std::mutex> lock(mtx);
            tasks.push(std::move(task));
        }
        cv.notify_one();
    }

    ~SimpleThreadPool() {
        {
            std::lock_guard<std::mutex> lock(mtx);
            stop = true;
        }
        cv.notify_all();  // Wake all workers to check stop flag

        for (auto& worker : workers) {
            worker.join();
        }
    }
};

int main() {
    SimpleThreadPool pool(4);

    for (int i = 0; i < 10; ++i) {
        pool.enqueue([i]{ std::cout << "Task " << i << "\n"; });
    }

    std::this_thread::sleep_for(std::chrono::seconds(1));
    // Destructor stops pool
}
```

What You Learned

Key Takeaways

- ✓ Condition Variable Overview
- ✓ The Wait-Notify Mechanism
- ✓ Why std::unique_lock is Required
- ✓ Notification Strategies

→ Swipe to learn →